

Container und virtuelle Maschinen – Isolationsmechanismen, Bedrohungsmodelle und Sicherheitsanalyse

IT-Sicherheit Übung – Thema 12, Hochschule Karlsruhe

1st Simon Klaes
Hochschule Karlsruhe
Karlsruhe, Deutschland
klsi1012@h-ka.de
[Abschnitt I](#)

2nd Radek Pekul
Hochschule Karlsruhe
Karlsruhe, Deutschland
pera1011@h-ka.de
[Abschnitt II](#)

3rd Joel Maxeiner
Hochschule Karlsruhe
Karlsruhe, Deutschland
majo1064@h-ka.de
[Abschnitt III](#)

4th Pius Tebbe
Hochschule Karlsruhe
Karlsruhe, Deutschland
tepi1011@h-ka.de
[Abschnitt IV](#)

Zusammenfassung—Container und virtuelle Maschinen (VMs) sind die zentralen Bausteine moderner IT-Infrastrukturen, unterscheiden sich aber grundlegend in ihrer Isolationsstärke. Diese Arbeit vergleicht beide Konzepte hinsichtlich ihrer Sicherheitseigenschaften: Sie erläutert die fundamentalen Isolationsmechanismen (Namespaces, cgroups, Hypervisor-Virtualisierung), demonstriert in einer Live-Demo den Normalbetrieb beider Technologien sowie typische Fehlkonfigurationen, und diskutiert reale Escape-Szenarien und Best Practices. Ziel ist es, fundiert zu zeigen, wann welche Technologie sicherheitstechnisch angemessen ist.

Index Terms—Container, Docker, virtuelle Maschine, Hypervisor, Namespaces, cgroups, Isolation, Container-Escape, IT-Sicherheit

I. GRUNDLAGEN DER ISOLATION

A. Motivation und Bedrohungsmodell

Sobald mehrere Workloads auf derselben Hardware laufen, stellt sich die Frage, *welche Komponente bei einem Ausbruch versagen darf*. Beim **Container-Bedrohungsmodell** wird angenommen, dass der Angreifer bereits Code im Container ausführt – etwa über eine kompromittierte Webanwendung oder ein böses Image – und versucht, den Host oder Nachbarcontainer zu übernehmen. Beim **VM-Bedrohungsmodell** wird sogar angenommen, dass der Gast vollständig kompromittiert ist (Root im Gast-Kernel); dennoch sollen Host und andere Gäste sicher bleiben. Container schützen damit *vor versehentlichen Konflikten zwischen vertrauenswürdigen Workloads*, VMs dagegen *vor bösen Mandanten* und sind deshalb Standard für Multi-Tenant-Clouds wie AWS EC2 oder Azure VMs [1], [3].

B. Virtualisierungskonzepte: Hypervisor-Typen

Eine VM simuliert einen kompletten Computer mit eigenem Kernel, eigener Hardware-Sicht und eigenem Bootloader. Der Hypervisor ist die Schicht, die diese Trennung erzwingt:

- **Typ 1 (Bare-Metal)**: läuft direkt auf der Hardware, ohne Host-OS dazwischen. Beispiele: KVM, Xen, VMware

ESXi, Microsoft Hyper-V. Geringe Trusted Computing Base (TCB) und produktiv bevorzugt.

- **Typ 2 (Hosted)**: läuft als Anwendung auf einem Betriebssystem. Beispiele: VirtualBox, VMware Workstation, Parallels Desktop, UTM.

Hardware-Erweiterungen (Intel VT-x, AMD-V, ARM Virtualization Extensions) führen einen privilegierten CPU-Modus ein („Ring –1“ bzw. EL2 auf ARM). Privilegierte Operationen des Gasts lösen einen *VM Exit* aus, den der Hypervisor behandelt. EPT/NPT übersetzt Gast-physikalische zu Host-physikalischen Adressen in Hardware. Die Schnittstelle zwischen Gast und Host besteht damit nur aus wenigen Hypercalls und paravirtualisierten Geräten (virtio) – eine deutlich kleinere Angriffsfläche als das Linux-Syscall-Interface [3].

C. Container-Grundlagen: Namespaces, Cgroups, Kernel-Sharing

Container sind technisch **keine eigene Technologie**, sondern eine Kombination bestehender Linux-Kernel-Features. Sie sind im Wesentlichen isolierte Prozesse, die denselben Kernel mit dem Host teilen.

Namespaces schaffen pro Container eine eigene Sicht auf Kernel-Ressourcen: `PID` (eigene Prozessbäume), `NET` (eigener Netzwerkstack), `MNT` (eigene Mount-Points), `IPC`, `UTS` (Host-name), `USER` (UID-Mapping), `CGROUP` und `TIME`. Namespaces beschränken *Sichtbarkeit*, nicht welche Syscalls ausgeführt werden dürfen.

Control Groups (cgroups) sind orthogonal dazu und beschränken *Ressourcenverbrauch* (CPU, Speicher, Block-I/O, Anzahl Prozesse). Ohne cgroups könnte ein einzelner Container per Fork-Bombe das Host-System lahmlegen.

Ergänzend reduzieren weitere Mechanismen die Angriffsfläche: **Capabilities** zerlegen die Root-Rechte in ca. 40 Einzelrechte (Container erhalten standardmäßig nur ca. 14), **Seccomp-BPF** filtert Syscalls (Docker blockiert per Default ca. 60 von ca. 330), und **LSMs** wie AppArmor oder SELinux setzen Mandatory Access Control durch.

D. Vertrauensgrenzen und Vergleich

Der entscheidende Unterschied ist die Frage, was bei einem Ausbruch versagen muss. Bei Containern ist die Vertrauensgrenze der **geteilte Linux-Kernel** – jede ausnutzbare Kernel-Schwachstelle ist potenziell ein Container-Escape. Die TCB umfasst ca. 30 Millionen Zeilen Code (gesamter Kernel inkl. Treiber, Syscall-Interface, Dateisystemen). Bei VMs ist die Vertrauensgrenze der **Hypervisor**; die TCB im Hot-Path liegt z. B. bei KVM bei rund 100 000 Zeilen Code – zwei bis drei Größenordnungen weniger Angriffsfläche [1], [3]. Tabelle I fasst die wesentlichen Dimensionen zusammen.

Tabelle I
VERGLEICH DER ISOLATIONS GARANTIE

Dimension	Container	VM
Isolationsgrenze	Kernel (geteilt)	Hardware (Hypervisor)
Boot-Zeit	Millisekunden	Sekunden bis Minuten
Overhead	MB	Hunderte MB bis GB
Dichte/Host	Hunderte	Dutzende
Angriffsfläche	~330 Syscalls	Hypercalls + Geräte
TCB	~30 Mio. LOC	~100k LOC
Multi-Tenancy	Unzureichend	Standard

II. IMPLEMENTIERUNG UND NORMALBETRIEB

A. Ziel des Vergleichs

Zur praktischen Gegenüberstellung wird derselbe Dienst in zwei Umgebungen betrieben. Einmal läuft er als Docker-Container und einmal in einer virtuellen Maschine. Als Beispiel dient ein einfacher Nginx-Webserver, da er leicht einzurichten ist und der Zugriff über den Browser einfach geprüft werden kann.

Dabei werden der Ressourcenbedarf, der Netzwerkzugriff, die Systemrechte und die Grenzen der Isolation verglichen.

B. Docker Implementierung

Der Webserver wird in Docker mit begrenzten Ressourcen gestartet.

```
docker run -d --name nginx-demo --memory=256m  
--cpus=0.5 -p 8080:80 nginx
```

Der Dienst ist anschließend über `http://localhost:8080` erreichbar. Mit `docker ps` lässt sich prüfen, ob der Container läuft. Der Befehl `docker stats` zeigt den aktuellen CPU- und Speicherverbrauch. Weitere Informationen zur Netzwerkkonfiguration liefert `docker inspect`.

Container starten sehr schnell, weil kein vollständiges Betriebssystem gebootet werden muss. Sie teilen sich den Kernel mit dem Hostsystem und werden vor allem durch Namespaces und cgroups isoliert. Namespaces trennen zum Beispiel Prozesse, Netzwerk und Dateisystemansichten. Cgroups begrenzen die Nutzung von Ressourcen wie CPU und Arbeitsspeicher.

C. Implementierung in der virtuellen Maschine

Für die virtuelle Maschine wird beispielsweise eine Ubuntu-VM in VirtualBox verwendet. Im Gegensatz zum Container besitzt die VM ein eigenes Betriebssystem mit eigenem Kernel. Innerhalb der VM wird Nginx installiert und gestartet.

```
sudo apt update  
sudo apt install nginx -y  
sudo systemctl enable nginx  
sudo systemctl start nginx
```

Der Status des Dienstes kann mit folgendem Befehl geprüft werden.

```
systemctl status nginx
```

Im NAT-Modus ist die VM nicht direkt von außen erreichbar. Über eine Portweiterleitung, zum Beispiel von Host-Port 8081 auf VM-Port 80, kann der Webserver über `http://localhost:8081` vom Host aus geöffnet werden.

Für die Analyse können in der VM mehrere Befehle genutzt werden. `ip a` zeigt die Netzwerkkonfiguration. Mit `ps aux | grep nginx` wird geprüft, ob Nginx-Prozesse laufen. `free -h` zeigt die aktuelle RAM-Nutzung und mit `top` können laufende Prozesse sowie CPU- und Speicherverbrauch beobachtet werden.

D. Vergleich der beiden Varianten

Container sind besonders effizient. Sie benötigen wenig Speicher, starten schnell und lassen sich mit Optionen wie `-memory` und `-cpus` gezielt begrenzen. Ihre Isolation basiert jedoch auf dem gemeinsamen Host-Kernel. Dadurch ist sie schwächer als bei einer virtuellen Maschine.

Virtuelle Maschinen benötigen mehr RAM, mehr Speicherplatz und mehr Startzeit. Dafür bieten sie eine stärkere Trennung zwischen Host und Gast. Root-Rechte innerhalb der VM gelten normalerweise nur im Gastbetriebssystem und nicht direkt auf dem Host. Ein Ausbruch aus der VM würde in der Regel eine Schwachstelle im Hypervisor erfordern.

Auch beim Netzwerk gibt es Unterschiede. Docker verwendet standardmäßig ein Bridge-Netzwerk mit Portweiterleitung. Die VM kann dagegen im NAT-Modus oder im Bridged-Modus betrieben werden. NAT ist stärker abgeschottet, während die VM im Bridged-Modus wie ein eigenes Gerät im lokalen Netzwerk erscheint.

Tabelle II
VERGLEICH VON CONTAINER- UND VM-IMPLEMENTIERUNG

Kriterium	Container	VM
Kernel	Gemeinsamer Host-Kernel	Eigener Gast-Kernel
Ressourcen	cgroups und Docker-Limits	Zuweisung über Hypervisor
Netzwerk	Bridge-Netzwerk mit Port-Mapping	NAT oder Bridged-Modus
Systemrechte	Root im Container eingeschränkt	Root nur im Gastbetriebssystem
Isolation	Leichtgewichtig, aber kernelabhängig	Stärker durch Hypervisor
Effizienz	Schneller Start, geringer Verbrauch	Höherer Ressourcenbedarf

E. Fazit

Der Normalbetrieb zeigt den zentralen Unterschied zwischen beiden Ansätzen. Container sind schneller und ressourcenschonender, bieten aber eine schwächere Isolation, da sie den Host-Kernel teilen. Virtuelle Maschinen sind weniger effizient, trennen Host und Gast jedoch stärker durch den Hypervisor und den eigenen Kernel.

Für sichere Container-Deployments sollten nur notwendige Ports freigegeben, Ressourcenlimits gesetzt und riskante Optionen wie `--privileged` vermieden werden.

III. FEHLKONFIGURATIONEN UND SCHWÄCHUNG DER ISOLATIONSGRENZEN

In diesem Abschnitt wird gezeigt, wie gezielte Fehlkonfigurationen die zuvor beschriebenen Isolationsmechanismen von Containern und virtuellen Maschinen aushebeln können. Alle Demonstrationen wurden in einer isolierten KVM-Umgebung (Ubuntu 24.04 LTS auf Proxmox VE) durchgeführt. Weder produktive noch fremde Systeme wurden dabei berührt. Als Laufzeit diente Docker mit `runc ≥ 1.1.12`, sodass die bekannte Escape-Schwachstelle CVE-2024-21626 ausgeschlossen ist. Jede Fehlkonfiguration wurde zunächst mit einer korrekt konfigurierten Umgebung verglichen, um die Auswirkungen sichtbar zu machen. Die zugehörigen Bildschirmaufnahmen finden sich im Anhang.

A. Demo 1: Privilegierter Container (`--privileged`)

Als erste Fehlkonfiguration wird die Aufhebung der Container-Isolation durch den Schalter `--privileged` demonstriert. Ohne diesen Schalter (Normalbetrieb) startet Docker Container mit einem reduzierten Capability-Set, das sicherheitskritische Rechte wie `CAP_SYS_ADMIN` ausschließt. Ist der Schalter `--privileged` gesetzt, werden diese Beschränkungen aufgehoben, die Seccomp- und AppArmor-Profilen deaktiviert und der Zugriff auf sämtliche Host-Geräte gewährt. Der direkte Vergleich desselben Befehls macht dies sichtbar: Während ein normaler Container beim Zugriff auf das über `--pid=host` erreichbare Host-Wurzelverzeichnis (`/proc/1/root/etc/shadow`) mit einer Zugriffsverweigerung abgewiesen wird, liest der privilegierte Container die Passwort-Hashes des Hosts im Klartext aus (Anhang, Abb. 1). Damit sind die Capability- und die Geräte-Isolation verletzt; ein Angreifer im Container besitzt effektiv Root-Rechte auf dem Host. Als Gegenmaßnahme ist auf `--privileged` zu verzichten und das Capability-Set mit `--cap-drop=ALL` zu leeren, um nur die tatsächlich benötigten Rechte einzeln zu vergeben.

B. Demo 2: Bind-Mount und Dateisystem-Isolation

In der zweiten Demonstration wird die Aufhebung der Dateisystem-Isolation durch einen Bind-Mount gezeigt. Normalerweise ist das Container-Dateisystem von dem des Hosts getrennt, doch mit einem Bind-Mount kann ein Host-Verzeichnis direkt in den Container eingebunden werden. Im Normalbetrieb zeigt der Container nur die gesperrten Systemkonten seines eigenen Images; nach dem Bind-Mount erscheinen

hingegen die realen Host-Konten samt gültigem SHA-512-Hash des regulären Benutzers (Anhang, Abb. 2). Ein anschließender Schreibzugriff auf `/host-etc/hosts` ist nach dem Verlassen des Containers auch auf dem Host nachweisbar (Anhang, Abb. 3) und belegt die bidirektionale Aufhebung der Isolation. Mit Schreibrecht auf `/etc` ließe sich ein zusätzlicher Root-Benutzer anlegen oder ein SSH-Schlüssel hinterlegen. Abzusichern ist dies, indem nur benötigte Pfade und möglichst schreibgeschützt (`:ro`) eingebunden werden.

C. Demo 3: Deaktivierung von Seccomp und AppArmor

Als dritte Fehlkonfiguration wird die Deaktivierung der Seccomp- und AppArmor-Profilen demonstriert. Standardmäßig blockiert das Docker-Seccomp-Profil etwa vierundvierzig Systemaufrufe, die für Kernel-Exploits nutzbar sind. Der Schalter `--security-opt seccomp=unconfined` schaltet diesen Filter ab, was unmittelbar am Wechsel des Seccomp-Status von 2 auf 0 im `/proc/1/status` des Container-Prozesses erkennbar ist (Anhang, Abb. 4). Zuvor gefilterte Aufrufe wie `unshare` werden danach ausführbar. Das Abschalten allein bewirkt zwar keinen Ausbruch in den Container ein und umgeht so die Dateisystem-Trennung, ohne dass eine Software-Schwachstelle ausgenutzt werden müsste, es öffnet jedoch die Tür für die Ausnutzung von Kernel-Schwachstellen und unterläuft so das Prinzip der gestaffelten Verteidigung. Die Standardprofile sollten daher nie pauschal deaktiviert, sondern bei Bedarf durch eng gefasste eigene Profile ersetzt werden.

D. Demo 4: Netzwerkkonfiguration der VM

Die letzte Fehlkonfiguration betrifft die Netzwerkkonfiguration der virtuellen Maschine. Im Normalbetrieb ist die VM über eine interne Bridge mit dem Host verbunden, sodass sie keinen direkten Zugriff auf das produktive Netzwerk hat. Die Umstellung auf eine Bridge mit physischem Uplink ermöglicht der VM hingegen die direkte Kommunikation mit anderen Geräten im LAN. In der Demonstration war derselbe Dienst bei interner Bridge von außen nicht erreichbar, nach Umstellung auf eine Bridge mit Uplink jedoch von einem zweiten Gerät im LAN ansprechbar (Anhang, Abb. 5, 6). Hier wird die Netzwerk-Vertrauensgrenze verletzt: Eine kompromittierte VM kann als Ausgangspunkt für Angriffe auf weitere Netzwerkgeräte dienen, ohne dass der Hypervisor dies verhindert. Darum sind Test- und Demonstrationssysteme an einer internen Bridge ohne Uplink oder hinter NAT zu betreiben und durch Firewall-Regeln sowie Netzsegmentierung abzusichern.

Am Ende zeigt sich, dass Container anders als virtuelle Maschinen denselben Kernel wie der Host nutzen und ihre Isolation allein auf Software-Mechanismen beruht. Fällt einer dieser Mechanismen durch Fehlkonfiguration weg, ist der Weg zum Host entsprechend kurz.

IV. FAILURE- UND ESCAPE-SZENARIEN IM VERGLEICH

A. Failure-Szenarien

Ressourcen-Überlastung:

- Container: Container teilen sich den Host-Kernel und konkurrieren direkt um CPU/RAM. Wenn ein Container

plötzlich sehr viel CPU oder RAM verbraucht, kann er ohne strikte Begrenzung andere Container beeinträchtigen. Diese erhalten dann weniger CPU-Zeit oder Arbeitsspeicher, was als sogenanntes „Noisy-Neighbor“-Problem bezeichnet wird.

- Virtuelle Maschine: VMs wird beim Start eine feste Menge an CPU und RAM zugewiesen. Verbraucht die Anwendung in der VM zu viel, stürzt nur diese eine VM ab. Der Host und andere VMs bleiben unberührt.

Kernel-Crash:

- Container: Verursacht eine Anwendung in einem Container einen kritischen Fehler, der den Kernel zum Absturz bringt, sind alle Container auf dem betroffenen Host betroffen. Da kein eigener Kernel pro Container existiert, kann der Ausfall nicht auf einen einzelnen Container begrenzt werden.
- Virtuelle Maschine: Bringt eine Anwendung den Kernel zum Absturz, stürzt nur das Gast-Betriebssystem dieser einen VM ab. Der Hypervisor läuft weiter, und alle anderen VMs merken nichts davon.

B. Escape-Szenarien

Ausbruch über den Kernel bzw. Hypervisor:

- Container: Da Container direkt mit dem Kernel des Host-Systems kommunizieren, reicht eine Sicherheitslücke im Kernel (z. B. durch Systemaufrufe/Syscalls), um einen Container-Escape zu ermöglichen. Ist der Angreifer einmal auf dem Host-System, hat er Zugriff auf alle dort laufenden Container.
- Virtuelle Maschine: Bei VMs ist für einen Escape eine Schwachstelle im Hypervisor erforderlich. Dieser bildet eine zusätzliche Isolationsebene zwischen Gast und Host und stellt nur eine stark begrenzte, hardwarenahe Schnittstelle bereit. Dadurch sind Hypervisor-Escapes technisch deutlich komplexer und seltener. Ein Beispiel ist CVE-2019-2525 (Oracle VirtualBox), eine Schwachstelle in der Core-Komponente, die (oft in Kombination mit anderen Lücken) einen Escape ermöglichen konnte.

Fehlkonfiguration als Einfallstor:

- Container: Container sind besonders anfällig für Fehlkonfigurationen. Werden Container beispielsweise als root-Benutzer mit erweiterten Rechten oder mit Zugriff auf den Docker-Socket gestartet, kann ein Angreifer aus dem Container heraus privilegierte Aktionen auf dem Host-System durchführen. In solchen Fällen entsteht ein Escape häufig nicht durch eine Software-Sicherheitslücke, sondern durch fehlerhafte Konfiguration.
- Virtuelle Maschine: Auch bei virtuellen Maschinen können Fehlkonfigurationen ein Sicherheitsrisiko darstellen, beispielsweise durch falsch konfigurierte Management-Schnittstellen, gemeinsam eingebundene Ordner oder unzureichende Netzwerksegmentierung. Selbst bei Administratorrechten innerhalb der VM bleibt die Isolation durch den Hypervisor jedoch in der Regel bestehen, sodass ein direkter Zugriff auf das Host-System deutlich erschwert wird.

PRAKTISCHE SICHERHEITS AUSWIRKUNGEN IN REALEN DEPLOYMENT-SZENARIEN

• Multi-Tenant-Umgebungen (z. B. öffentliche Cloud):

In Umgebungen, in denen mehrere Kunden dieselbe physische Infrastruktur nutzen, gelten virtuelle Maschinen als sicherer, da der Hypervisor eine zusätzliche Isolationsebene bereitstellt. Gelingt einem Angreifer jedoch ein Container-Escape, kann er unter Umständen auf Ressourcen anderer Container oder sogar auf den Host zugreifen. Zum Beispiel könnte dies dazu führen, dass sensible Kundendaten ausgelesen, Anwendungen manipuliert oder weitere Systeme kompromittiert werden. Ein erfolgreicher Angriff hätte damit potenziell Auswirkungen auf mehrere Mandanten gleichzeitig und könnte erhebliche finanzielle sowie rechtliche Folgen nach sich ziehen.

• Entwicklung und Produktion:

In Entwicklungsumgebungen werden Container häufig mit erweiterten Rechten oder vereinfachten Konfigurationen betrieben, um Entwicklungsprozesse zu beschleunigen. Werden solche Konfigurationen versehentlich in die Produktionsumgebung übernommen, kann ein Angreifer diese ausnutzen, um privilegierten Zugriff auf das System zu erlangen. Ein mögliches Szenario ist die Kompromittierung einer Webanwendung innerhalb eines Containers, von der aus sich der Angreifer aufgrund überhöhter Rechte Zugriff auf den Host verschafft. Dadurch könnten Datenbanken manipuliert, vertrauliche Kundendaten entwendet oder geschäftskritische Dienste außer Betrieb gesetzt werden. Für das betroffene Unternehmen kann dies zu Produktionsausfällen, finanziellen Schäden, Datenschutzverletzungen und einem Vertrauensverlust bei Kunden führen.

• CI/CD-Pipelines und Supply-Chain-Angriffe:

Container werden häufig über öffentliche Registries oder interne Image-Repositories automatisiert in Build- und Deployment-Prozesse eingebunden. Wird ein kompromittiertes Container-Image in die Pipeline eingeschleust, kann sich Schadcode automatisiert in zahlreiche Systeme verbreiten. Ein möglicher Angriff wäre, dass ein Angreifer ein scheinbar harmloses Basis-Image manipuliert, das von einem Unternehmen für mehrere Anwendungen verwendet wird. Nach dem automatischen Einspielen des Images in die CI/CD-Pipeline wird der Schadcode in allen neu erstellten Container-Instanzen ausgerollt. Dadurch könnte der Angreifer vertrauliche Kundendaten abgreifen, Zugangsdaten auslesen oder eine Hintertür installieren, über die später weitere Systeme kompromittiert werden. Da sich die Verteilung über automatisierte Prozesse vollzieht, bleibt der Angriff möglicherweise über längere Zeit unentdeckt und kann eine große Anzahl von Produktionssystemen gleichzeitig betreffen.

V. BEST PRACTICES FÜR SICHERE KONFIGURATION

A. Container

- Ressourcenlimits setzen, um Noisy-Neighbor-Probleme zu vermeiden.

- Keine privilegierten Container und möglichst Rootless Container in Produktionsumgebungen verwenden.
- Seccomp, AppArmor oder SELinux aktivieren, um Kernel-Escapes zu erschweren.
- Read-Only RootFS und Volumes nutzen, um Dateisystem-Schäden zu begrenzen.
- Regelmäßige Updates von Kernel und Container-Runtime reduzieren das Risiko bekannter Container-Escapes.

B. Virtuelle Maschinen

- CPU-/RAM-Zuweisung sinnvoll konfigurieren, um Überlastung zu vermeiden.
- Hypervisor regelmäßig patchen, um Escape-Schwachstellen zu schließen.
- Kein unnötiges Geräte-Passthrough aktivieren.
- Snapshots und Backups gegen Dateisystem-Fehler einsetzen.
- Sicherheitsupdates und gegen Side-Channel-Angriffe einspielen.

C. Gesamtfazit

Container bieten hohe Effizienz und Flexibilität, weisen jedoch aufgrund des gemeinsam genutzten Kernels geringere Isolationsgarantien auf als virtuelle Maschinen. Virtuelle Maschinen verursachen zwar höhere Ressourcen-Overheads, bieten jedoch insbesondere in Multi-Tenant- und sicherheitskritischen Szenarien stärkere Sicherheitsgrenzen. In der Praxis hängt die Wahl daher stark vom Bedrohungsmodell und den Sicherheitsanforderungen des jeweiligen Deployment-Szenarios ab.

HINWEIS ZUR KI-NUTZUNG

Bei der Recherche und der Erstellung dieses Dokuments wurden KI-gestützte Werkzeuge (Anthropic Claude) zur Strukturierung von Inhalten, zum Erklären von Fachbegriffen und zur sprachlichen Glättung eingesetzt. Sämtliche inhaltlichen Aussagen wurden von den Autorinnen und Autoren geprüft und gegen die Lehrgangs- und Referenzquellen abgeglichen.

LITERATUR

- [1] M. Souppaya, J. Morello und K. Scarfone, „Application Container Security Guide“, NIST Special Publication 800-190, National Institute of Standards and Technology, Gaithersburg, MD, USA, 2017.
- [2] A. Y. Wong, E. G. Chekole, M. Ochoa und J. Zhou, „Threat Modeling and Security Analysis of Containers: A Survey“, arXiv:2111.11475, 2021.
- [3] U. Gupta, „Comparison between Security Majors in Virtual Machine and Linux Containers“, arXiv:1507.07816, 2015.
- [4] MITRE, „CVE-2019-5736: runc Container Breakout“, 2019.
- [5] M. Kellermann, „The Dirty Pipe Vulnerability (CVE-2022-0847)“, 2022.
- [6] CrowdStrike, „VENOM Vulnerability (CVE-2015-3456)“, 2015.

ANHANG

```
vm@vm:~$ sudo docker run --rm --pid=host ubuntu cat /proc/1/root/etc/shadow
[sudo] password for vm:
cat: /proc/1/root/etc/shadow: Permission denied
vm@vm:~$ sudo docker run --rm --privileged --pid=host ubuntu cat /proc/1/root/etc/shadow
root:*:20494:0:99999:7:::
daemon:*:20494:0:99999:7:::
bin:*:20494:0:99999:7:::
sys:*:20494:0:99999:7:::
sync:*:20494:0:99999:7:::
games:*:20494:0:99999:7:::
man:*:20494:0:99999:7:::
lp:*:20494:0:99999:7:::
mail:*:20494:0:99999:7:::
news:*:20494:0:99999:7:::
uucp:*:20494:0:99999:7:::
proxy:*:20494:0:99999:7:::
www-data:*:20494:0:99999:7:::
backup:*:20494:0:99999:7:::
list:*:20494:0:99999:7:::
irc:*:20494:0:99999:7:::
_apt:*:20494:0:99999:7:::
nobody:*:20494:0:99999:7:::
systemd-networkd:!*:20494:0:0:0:
systemd-timesyncd:!*:20494:0:0:0:
dhcpcd:!*:20494:0:0:0:
messagebus:!*:20494:0:0:0:
systemd-resolved:!*:20494:0:0:0:
pollinate:!*:20494:0:0:0:
polkitd:!*:20494:0:0:0:
usbmux:!*:20599:0:0:0:
vm:!
sshd:!*:20599:0:0:0:
dnsmasq:!*:20599:0:0:0:
```

Abbildung 1. Demo 1 – Privilegierter Container. Oben: normaler Container, Zugriff auf `/proc/1/root/etc/shadow` verweigert. Unten: privilegierter Container, Host-Passwort-Hashes lesbar.


```

[vm@vm:~]$ sudo docker run --rm ubuntu cat /etc/shadow
root:*:20564:0:99999:7:::
daemon:*:20564:0:99999:7:::
bin:*:20564:0:99999:7:::
sys:*:20564:0:99999:7:::
sync:*:20564:0:99999:7:::
games:*:20564:0:99999:7:::
man:*:20564:0:99999:7:::
lp:*:20564:0:99999:7:::
mail:*:20564:0:99999:7:::
news:*:20564:0:99999:7:::
uucp:*:20564:0:99999:7:::
proxy:*:20564:0:99999:7:::
www-data:*:20564:0:99999:7:::
backup:*:20564:0:99999:7:::
list:*:20564:0:99999:7:::
irc:*:20564:0:99999:7:::
_apt:*:20564:0:99999:7:::
nobody:*:20564:0:99999:7:::
ubuntu!:20564:0:99999:7:::
[vm@vm:~]$ sudo docker run --rm -it -v /etc:/host-etc ubuntu bash
root@9e01459b8b9a:/# cat /host-etc/shadow
root:*:20494:0:99999:7:::
daemon:*:20494:0:99999:7:::
bin:*:20494:0:99999:7:::
sys:*:20494:0:99999:7:::
sync:*:20494:0:99999:7:::
games:*:20494:0:99999:7:::
man:*:20494:0:99999:7:::
lp:*:20494:0:99999:7:::
mail:*:20494:0:99999:7:::
news:*:20494:0:99999:7:::
uucp:*:20494:0:99999:7:::
proxy:*:20494:0:99999:7:::
www-data:*:20494:0:99999:7:::
backup:*:20494:0:99999:7:::
list:*:20494:0:99999:7:::
irc:*:20494:0:99999:7:::
_apt:*:20494:0:99999:7:::
nobody:*:20494:0:99999:7:::
systemd-network:!:20494:0:99999:7:::
systemd-timesync:!:20494:0:99999:7:::
dhcpcd:!:20494:0:99999:7:::
messagebus:!:20494:0:99999:7:::
systemd-resolve:!:20494:0:99999:7:::
pollinate:!:20494:0:99999:7:::
polkitd:!:20494:0:99999:7:::
usbmux:!:20599:0:99999:7:::
vm:$
sshd:!:20599:0:99999:7:::
dnsmasq:!:20599:0:99999:7:::
root@9e01459b8b9a:/# cat /host-etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534:/:nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:998:998:systemd Network Management:/:/usr/sbin/nologin
systemd-timesync:x:997:997:systemd Time Synchronization:/:/usr/sbin/nologin
dhcpcd:x:100:65534:DHCP Client Daemon,,,:/usr/lib/dhcpcd:/bin/false
messagebus:x:101:102:/:nonexistent:/usr/sbin/nologin
systemd-resolve:x:992:992:systemd Resolver:/:/usr/sbin/nologin
pollinate:x:102:1:/:var/cache/pollinate:/bin/false
polkitd:x:991:991:User for polkitd:/:/usr/sbin/nologin
usbmux:x:103:46:usbmux daemon,,,:/var/lib/usbmux:/usr/sbin/nologin
vm:x:1000:1000:Joel:/home/vm:/bin/bash
sshd:x:104:65534:/:run/sshd:/usr/sbin/nologin
dnsmasq:x:999:65534:dnsmasq:/var/lib/misc:/usr/sbin/nologin

```

Abbildung 2. Demo 2 – Bind-Mount. Der Container liest über /host-etc/shadow die Host-Hashes; der Hash des Benutzerkontos wurde geschwärzt.

```

[root@9e01459b8b9a:/# echo "203.0.113.66  evil.example" >> /host-etc/hosts
[root@9e01459b8b9a:/# tail -n 3 /host-etc/hosts
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
203.0.113.66  evil.example
[root@9e01459b8b9a:/# exit
exit
[vm@vm:~$ tail -n 3 /etc/hosts
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
203.0.113.66  evil.example
vm@vm:~$ █

```

Abbildung 3. Demo 2 – Schreibnachweis. Die aus dem Container eingefügte Zeile erscheint auf dem Host in /etc/hosts.

```

[vm@vm:~$ sudo docker run --rm ubuntu bash -c 'grep Seccomp /proc/1/status'
Seccomp:          2
Seccomp_filters:   1
[vm@vm:~$ sudo docker run --rm --security-opt seccomp=unconfined ubuntu \
[  bash -c 'grep Seccomp /proc/1/status'
Seccomp:          0
Seccomp_filters:   0
[vm@vm:~$ sudo docker run --rm ubuntu \
[  bash -c 'unshare --user --map-root-user echo OK 2>&1 || echo blockiert'
unshare: unshare failed: Operation not permitted
blockiert
[vm@vm:~$ sudo docker run --rm --security-opt seccomp=unconfined ubuntu \
[  bash -c 'unshare --user --map-root-user echo "unshare OK - kein Filter"'
unshare OK - kein Filter
vm@vm:~$ █

```

Abbildung 4. Demo 3 – Seccomp-Status: aktiver Filter (2) im Normalbetrieb gegenüber deaktiviertem Filter (0) bei seccomp=unconfined.


```
PING 192.168.2.41 (192.168.2.41): 56 data bytes
Request timeout for icmp_seq 0
Request timeout for icmp_seq 1
Request timeout for icmp_seq 2
Request timeout for icmp_seq 3
ping: sendto: No route to host
Request timeout for icmp_seq 4
```

Abbildung 5. Demo 4 – Netzwerk. Keine Erreichbarkeit des Dienstes von einem zweiten LAN-Gerät. Ping.

```
PING 192.168.2.41 (192.168.2.41): 56 data bytes
64 bytes from 192.168.2.41: icmp_seq=0 ttl=64 time=4.307 ms
64 bytes from 192.168.2.41: icmp_seq=1 ttl=64 time=4.945 ms
64 bytes from 192.168.2.41: icmp_seq=2 ttl=64 time=5.154 ms
```

Abbildung 6. Demo 4 – Netzwerk. Erreichbarkeit des Dienstes von einem zweiten LAN-Gerät. Ping.